

LISTA DE EXERCÍCIOS 3.10

1. Explique brevemente os esquemas de alocação de *frames* fixado igualitário e proporcional.

Resposta: o esquema de alocação de *frames* igualitário procura dividir a quantidade de *frames* disponível de maneira igualitária entre os processos existentes no sistema, salvaguardando-se os *frames* necessários para o armazenamento do SO. Esse esquema de alocação pode gerar desperdício de memória (i.e. fragmentação) tendo em vista que não leva em conta o tamanho dos processos. No esquema de alocação de *frames* proporcional, a memória é alocada de acordo com o tamanho dos processos, de modo a se favorecer de maneira mais dinâmica a ocorrência de multiprogramação

2. O que vem a ser *thrashing*?

Resposta: *thrashing* é um efeito de troca frequente de páginas em memória (*page in/out*), devido ao fato de um processo não possuir páginas suficientes para sua representação adequada. Com isso, ao ocorrer uma *page fault* e trazer-se um conteúdo para memória mediante a substituição de algum *frame*, este *frame* precisa ser trazido de volta rapidamente para utilização.

3. O que vem a ser o modelo de localidade? Qual é a relação desse modelo e a ocorrência de *thrashing*.

Resposta: a localidade é a representação do contexto corrente de um determinado processo. Esse contexto pode ser a função corrente em execução ou a *thread* corrente. Durante sua execução, um processo migra de uma localidade para outra. Ao se mudar de localidade, é possível que as páginas correntes que estão carregadas para o processo não sejam suficiente (ou sejam totalmente desnecessárias) de modo que novas páginas precisam ser carregadas. Caso não haja memória suficiente, e considerando uma mudança de localidade frequente, é possível que o processo entre em *thrashing* ao ficar realizando *page in/out* frequente para atender as mudanças de localidade necessárias a sua execução.

4. O que vem a ser o modelo de conjunto de trabalho?

Resposta: por esse modelo, tenta-se estimar qual é a demanda total de *frames* para um processo. Para tanto, definem-se “janelas” capazes de conter um conjunto de páginas necessário para representar uma determinada localidade. Considerando-se todas as localidades que representam um processo, a memória deve ser suficiente para atender essa demanda (i.e. para o estabelecimento dos n conjuntos de trabalho representativos das localidades sem a geração de *page faults* constantes – *thrashing*). Caso a demanda de um processo seja muito maior do que a memória disponível, deve-se considerar a realização do *swap out* de outro processo, de modo a atender-se a demanda corrente de maneira adequada.

5. O que vem a ser o modelo PFF (*page-fault frequency* – PFF)? Qual é a relação de PFF com o conjunto de trabalho?

Resposta: o modelo PFF é uma política de substituição local de páginas a qual verifica a frequência de ocorrência de *page faults*. Se a taxa de ocorrência de *page faults* é muito baixa para um dado processo, este perde *frames* em memória. Caso a taxa de ocorrência de *page faults* seja muito alta, este ganha *frames* em memória, de modo a se estabelecer um equilíbrio entre a ocupação de memória e a não ocorrência de *thrashing* para um dado processo. Assim, pode-se utilizar o PFF como medida para o estabelecimento do tamanho do conjunto de trabalho.

6. O carregamento de arquivos em memória representa vantagem? Justifique sua resposta.

Resposta: sim. Com isso simplifica-se e acelera-se o acesso ao arquivo, devido a realização de

I/O de arquivo em memória ao invés de uso de chamadas de sistema *read()* e *write()*. Isso também admite que vários processos mapeiem o mesmo arquivo, permitindo que páginas em memória sejam compartilhadas (desde que se respeitem as regras de COW).

7. Considerando-se a alocação de memória de kernel, explique como funciona a alocação de memória via sistema *buddy*.

Resposta: o sistema *buddy* aloca memória a partir de um segmento de tamanho fixo consistindo de páginas fisicamente contíguas. Para tanto, utiliza um modelo de alocação baseado em potência de 2. Dado um bloco de memória livre, as requisições são satisfeitas pelo valor em potência de 2 mais próxima. Para alcançá-lo, a requisição é arredondada para a próxima potência de 2 mais alta. Quando uma alocação menor do que o espaço de memória disponível é necessária, o bloco (*chunk*) corrente é dividido em dois *buddies* da próxima potência de 2 mais baixa. Esse processo continua até que um bloco de tamanho apropriado esteja disponível.